

Introduction à Spark

Programmation avec Spark et Scala

2022-2023

Sommaire

1. Introduction à Spark
 1. Définition
 2. Initialisation de Spark
 1. Les options de Spark-shell
 2. SparkContext
2. RDD & DataSet / DataFrame
3. Déploiement avec IntelliJ + Spark-submit
4. Lecture et Ecriture de données avec Spark
5. Industrialisation avec Oozie

Introduction à Spark

- **Définition**

- Apache Spark est un système de traitement distribué open source utilisé pour les traitements de données à grande échelle.
- Utilise, pour des requêtes rapides sur des données de toute taille, des processus comme :
 - **In-Memory Caching** : Traitement et Stockage des données intermédiaires en utilisant la mémoire (écriture en Mémoire plus rapide que l'écriture en Disk)
 - **Shuffle** : Mécanisme de re-Distribution/ re-Partition des données lors des opérations.

Introduction à Spark

- **Les composants de Spark**

Apache Spark Core

Spark SQL

Spark
Streaming

MLlib
(Machine
Learning)

GraphX
(Graph)

Introduction à Spark

● Initialisation

● Spark avec

- Scala => **spark-shell** [Options]
- Python => **pyspark** [Options]

● Les Options

- | | |
|--------------------------------|--|
| ● --name NAME | :name = nom de l'application |
| ● --master TYPE | :type = yarn, local, mesos...etc. |
| ● --driver-memory MEM | :mem = Driver memory (default 1G) |
| ● --num-executors NUM | :num = nombre d'executeurs à l'exécution |
| ● --executor-memory MEM | :mem = executor memory (default 1G) |
| ● --executor-cores NUM | :num = nombre de cores par executeur |

Exercice: Initialisation Spark

- **Ouvrez une session spark-shell avec les options suivantes :**
 - **Donner comme nom à l'application : 'Spark by « mon_nom »'**
 - **La gestion des ressources doit être gérée par Yarn**
 - **5 exécuteurs et chaque exécuteur aura une mémoire de 3G et 5 cores**
 - **Le driver doit avoir une mémoire de 4G**
 - **Vérifier au niveau de l'environnement Yarn, que toutes vos options ont été prise en compte.**

Introduction à Spark

- **Initialisation**

- **SparkSession**

- Point d'entrée principal d'une exécution Spark (Main)
- A l'initialisation d'une Session (ex: spark-shell), l'objet SparkSession est disponible comme avec le nom **spark**
 - Mais peut être réinitialisé :

```
import org.apache.spark.sql.SparkSession
val spark:SparkSession = SparkSession.builder()
    .master("yarn")
    .appName("mon application")
    .getOrCreate()
```

Exercice: Initialisation Spark

- **Dans la session ouverte tout à l'heure :**
 - **Réinitialisée la et changer le nom par 'Change Spark by « mon_nom »'**
 - Avec la methode **getAll** de spark.conf, afficher toutes les options
 - Avec la méthode **get("key")**, afficher le nom de votre application. Vous pourrez retrouver la clé au niveau des informations affichées avec la méthode **getAll**

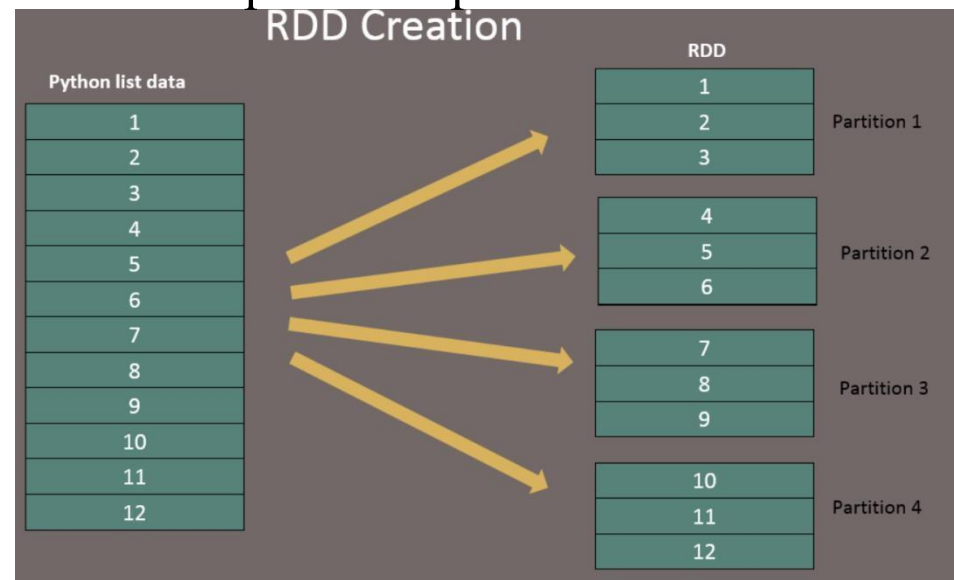
RDD & DataSet / DataFrame

● Les RDDs : Définition

- Resilient Distributed Dataset, sont des « *collections* » d'éléments partitionnées et distribuées à travers les nœuds du cluster
- C'est la structure fondamentale de base dans Spark
- Les RDD permettent d'effectuer des opérations parallèles en mémoire sur un cluster.

● Quelques caractéristiques

- Traitement en mémoire
- Lazy evaluation
- Immuables
- ...



RDD & DataSet / DataFrame

- **Les RDDs : Création**

- Il existe deux (2) manières de créer un RDD

- A partir d'une **collection** :

`spark.sparkContext.parallelize(collection)`

Exemple:

```
val fruits= Seq((1,"orange" ),(2,"pomme"),(3,"fraise"),(4,"citron"))
```

```
val fruitRDD = spark.sparkContext.parallelize(fruits )
```

- A partir d'une **source externe** :

`spark.sparkContext.textFile("cheminVersFichier")`

RDD & DataSet / DataFrame

- **Les RDDs : Opérations**

- De manière générale, les rdds supportent 2 types d'opérations:

- **1. Les Transformations :**

A partir d'un RDD, retourne un autre RDD après un traitement

- **map**(func)
- **filter**(func)
- **flatMap**(func)
- **groupByKey**([numPartitions])
- **reduceByKey**(func, [numPartitions])
- **sortByKey**([ascending], [numPartitions])
- **join**(otherDataset, [numPartitions])
- **coalesce**(numPartitions)
- **repartition**(numPartitions)

<https://spark.apache.org/docs/latest/rdd-programming-guide.html#transformations>

RDD & DataSet / DataFrame

- **Les RDDs : Opérations**

- De manière générale, les rdds supportent 2 types d'opérations:

- **2. Les Actions :**

A partir d'un RDD retourne une valeur

- **reduce**(*func*)
- **collect**()
- **count**()
- **first**()
- **countByKey**()
- **foreach**(*func*)

<https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>

Exercice RDDs

- Ouvrez une session spark
- Créer un RDD en se basant sur le fichier titanic.txt qui se trouve dans votre répertoire de base
- Supprimer la première ligne (entête)
- Créer un RDD contenant les TicketClass distincts
- Calculer le nombre de survivant par TicketClass
- Créer un RDD, ne contenant que les informations de Sex et Survived
- Quel est le nombre de femmes qui ont survécus dans titanic ?
- Créer un RDD ne contenant que les femmes.
- Le max age des survivants et des morts ?
- Quel est le nombre de ligne de titanic.txt ?
- Calculer le nombre de survivants et de morts en one-shot?
- Avec une combinaison de map et reduce, calculer l'âge total des personnes.

RDD & DataSet / DataFrame

- **Le DataFrame : Définition**

- Est une collections distribuées de données
- Est organisé en colonnes nommées. Il est équivalent à une table dans une base de données relationnelle.

Hive Data

Csv Data

Json Data

RDBMS Data

XML Data

Parquet Data

Cassandra Data

RDDs

Spark SQL

DataFrame

	Col1	Col2	Col3	
Row 1				
Row 2				
Row 3				

Lecture et Ecriture de
plusieurs formats de
données autorisées

RDD & DataSet / DataFrame

- **Le DataFrame : Création**
 - Il existe plusieurs manières de créer un DataFrame
 - A partir d'une **Collection**
 - A partir d'un **RDD**
 - A partir d'une **source de données Externes**

RDD & DataSet / DataFrame

- **Le DataFrame : Exemple de Création**

- **A partir d'un RDD**

- **Création**

```
val plats = Seq(("Thieb djeune", "2000"), ("Mafe", "3000"), ("Soupou kandja", "3500"))
```

```
val plats_rdd = spark.sparkContext.parallelize(plats)
```

```
val plats_dataframe = plats_rdd.toDF("Menu", "Prix")
```

- **Affichage**

```
plats_dataframe.show(false)
```

NB : La methode show() prend 2 arguments optionnelles :

- Truncate : true | false

Affiche les lignes du dataframe sans être tronquée

- Nb_row : Int

Le nombre de ligne du dataframe à afficher

RDD & DataSet / DataFrame

- **Le DataFrame : Exemple de Création**

- **A partir d'une collection**

- **Création en utilisant **ToDF****

```
import spark.implicits._
```

```
val plats_list = List(("Thieb djeune", "2000"), ("Mafe", "3000"), ("Soupou kandja", "3500"))
```

```
val plats_dataframe = plats_list .toDF("Menu", "Prix")
```

- **Création en utilisant **spark.createDataFrame****

```
val plats_list = List(("Thieb djeune", "2000"), ("Mafe", "3000"), ("Soupou kandja", "3500"))
```

```
val plats_dataframe = spark.createDataFrame(plats_list).toDF("Menu", "Prix")
```

RDD & DataSet / DataFrame

- **Le DataFrame : Exemple de Création**

- **A partir d'une source externe**

- **Création en utilisant `spark.read`.**

- `csv`
- `json`
- `avro`
- `parquet`
- `text / textFile`
- `orc`
- `jdbc`
- `....`

- **Création en à partir d'une table en utilisant `spark.sql("")`**

Exmple : `spark.sql("select * from ingestion.personne_tmp")`

Déploiement : IntelliJ + spark-submit

- **Le fichier « build.sbt »**

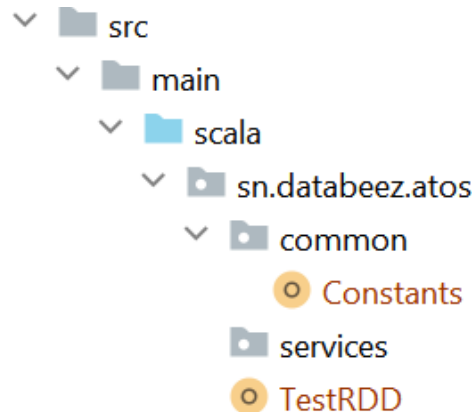
- Se trouve à la racine d'un projet spark scala
- Contient les dépendances à télécharger
- Exemple de fichier build.sbt =====>



build.sbt

- **Définition d'un « nice project structure »**

- Une classe doit se trouver dans un package
- Créer un sous package « common » qui contient les constants et variables / fonctions qui ne varient pas souvent
- Créer un sous package « services » qui contient les fonctions faisant appel à des services internes ou externes (ex: api)



Déploiement : IntelliJ + spark-submit

- **Création du « jar »**

- On met le code sur IntelliJ dans notre classe
- Ensuite, avec la commande sbt, nous créons le fichier « jar »

sbt package

- Après avoir exécuté la commande, un fichier « jar » est créé dans le répertoire :

target/scala- « version »/nomFichier.jar

- **Déplacer le « jar » au niveau du nœud edge, dans votre répertoire de base**

- A partir d'un terminal avec la commande **scp**
scp « chemin/fichier.jar » user@host:/cheminDestination
- A partir MobaXterm ou un autre Terminal : **Upload de fichier**

Déploiement : IntelliJ + spark-submit

- Exécuter le programme avec **spark-submit**

spark-submit [Options] fichier.jar [arguments]

- Les Options

- **--master** TYPE :type = yarn, local, mesos...etc.
- **--deploy-mode** MODE :mode = cluster, client
- **--class** CLASS :class = nom complet de la classe à exécuter (nom_package.nom_class)
- **--jars** JARS :jars = jars embarqués à l'exécution
- **--files** FILES :files = fichiers embarqués à l'exécution
- **--driver-memory** MEM :mem = Driver memory (default 1G)
- **--num-executors** NUM :num = nombre d'executeurs à l'exécution
- **--executor-memory** MEM :mem = executor memory (default 1G)
- **--executor-cores** NUM :num = nombre de cores par executeur
- **--queue** QUEUE_NAME :queue_name = nom de la queue utilisée

Déploiement : IntelliJ + spark-submit

- Exécuter le programme avec **spark-submit**

Exemple :

```
spark-submit --master yarn --deploy-mode cluster --num-executors 2 --class  
sn.databeez.atos.Test Test.jar
```

FIN
